

Assignment Package 1 – Lab 1

General instructions: How and when to submit and what to submit are explained on the submission page on Blackboard. This document describes the assignments.

For each assignment and, in some cases, sub-assignment, there is a .java file. The name of the file is stated in the respective assignment / sub-assignment.

NOTE! You need to use some of the classes found in Blackboard during the modules for weeks 1 and 2. You should not reprogram what is programmed in these classes; you should only use the classes. To do this, you need to download the classes to the folder where you write your programs for submission.

This information is found as part of the course description found in Blackboard:

- All assignment packages are divided into two parts called **Part 1** and **Part 2**. Part 1 is done to get a pass on the exam, while Part 2 is done to get bonus points for a higher grade (4, 5) on the second part of the exam (see the information below about the exam structure). That's it, Part 1 must be done to get a pass, while Part 2 is only done to collect bonus points for a higher grade on the exam.
- The **total** number of bonus points you can get by solving the tasks on Part 2 for each assignment package is **10**. That's it, you can collect half of the points for a higher grade on the exam by solving these Part 2 tasks. The bonus points you get for the exam are equal to the sum of the bonus points you have received from the solutions to the Part 2 tasks.

1. This task is about implementing a data type for students that can be used in study administrative contexts. In this context, a student has a name (first and last name), a personal identification number, and a number of university credits that he or she has completed.

Among other things, you want to be able to manage lists of students (for example, from a program or course) that you sort by name. But you may also want to sort students by completed university credits, for example, when it comes to determining which students have enough credits to be moved to the next year or to be able to be registered for the degree project.

(a) (4 points) In the file **Student.java** you will find the beginning of an implementation of a data type for students that you will complete. The file compiles, but the main method that tests your implementation gives the wrong output.

```
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/ equals  
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/ is false
```

```
[Student]/Studentdotter, She, pn: 0202028888, credits: 0.0/ equals  
[Student]/Studentsson, He, pn: 0302017777, credits: 0.0/ is false
```

Array with students:

```
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/  
[Student]/Studentdotter, She, pn: 0202028888, credits: 0.0/  
[Student]/Studentsson, He, pn: 0302017777, credits: 0.0/  
[Student]/Randomsson, Student, pn: 0302016666, credits: 0.0/
```

Sorted array:

```
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/  
[Student]/Studentdotter, She, pn: 0202028888, credits: 0.0/  
[Student]/Studentsson, He, pn: 0302017777, credits: 0.0/  
[Student]/Randomsson, Student, pn: 0302016666, credits: 0.0/
```

Added credits:

```
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/  
[Student]/Studentdotter, She, pn: 0202028888, credits: 0.0/  
[Student]/Studentsson, He, pn: 0302017777, credits: 0.0/  
[Student]/Randomsson, Student, pn: 0302016666, credits: 0.0/
```

What you need to do is program the methods **addCredits**, **credits**, **equals** and **compareTo**. The last-mentioned method is needed for the compiler to accept that Student implements Comparable<Student>

The class API describes what these methods should do:

```
public class Student implements Comparable<Student>{ ... }
```

// The constructor creates a student with the given
// social security number, first name and last name.
// The number of credits is set to 0.

// (NOTE: already implemented)
public Student(String pn, String fn, String ln)

// The method **toString** calculates a text with all the information about the student.
// The text looks like:

// [Student]/lastname, firstname, pn: pnumber, credits: acquired/
// (NOTE: already implemented)
public String toString()

/ The **equals** method calculates the boolean value of comparing for equality
// *this* student (with the instance variables) with the student *that* that is
// the method's argument.

// Two students are equal if they have the same first name, last name, and personnummer.

public boolean equals(Student that)

// The **compareTo** method calculates an integer that should be:

// -1 if this student is less than student that

// 0 if this student is equal (according to equals) to student that

// 1 if this student is greater than student that

// The order is determined by the lexicographical order of

// surnames, first names, and personnummers:

// Only if two students have the same last name do you look at the name

// and only if two students have the same last name and first name do you look at the

// personnummer to determine the order.

public int compareTo(Student that)

// The **addCredits** method is used to add credits that the student has earned.

// The value c is used to increment the value of the instance variable credits.

public void addCredits(double c)

// The **credits** method returns the value of the variable credits.

public double credits()

A correct implementation produces the following output (**you should not change the main method**. You should only implement the methods that are used in the correct way):

```
java Student
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/ equals
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/ is true

[Student]/Studentdotter, She, pn: 0202028888, credits: 0.0/ equals
[Student]/Studentsson, He, pn: 0302017777, credits: 0.0/ is false

Array with students:
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/
[Student]/Studentdotter, She, pn: 0202028888, credits: 0.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 0.0/
[Student]/Randomsson, Student, pn: 0302016666, credits: 0.0/

Sorted array:
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 0.0/
[Student]/Randomsson, Student, pn: 0302016666, credits: 0.0/
[Student]/Studentdotter, She, pn: 0202028888, credits: 0.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 0.0/

Added credits:
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.5/
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
```

(b) (1 point) In the file ***StudentCreditsComparator.java***, you should program a Comparator that compares students according to how many credits they have: students with fewer credits are smaller than students with more credits, students with the same number of credits are compared to regular students, according to last name, first name, and personnummer.

A correct implementation produces the following output (you should not change the main method!):

```
java StudentCreditsComparator
Array with students:
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.5/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
```

Sorted array:

```
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.5/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
```

(c) (1 point) In the file ***StudentAgeComparator.java***, you should program a Comparator that compares students with respect to their personnummer: students with a higher personnummer are considered smaller (younger) than students with a lower personnummer.

A correct implementation produces the following output. In this case, ***you must complete the main method to sort and print the sorted array.***

```
java StudentAgeComparator
Array with students:
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.5/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
```

Sorted array:

```
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.5/
```

Tips: There is no method in the Student class to find a student's personnummer. ***You should not modify the Student class.*** Keep in mind that the ***toString*** method produces a text that contains the personnummer: use that to determine a student's personnummer.

(d) (1 point) Write a program **SortStudents** that reads students from standard input and generates a sorted output. The program should be a class **SortStudents** in a file **SortStudents.java**. The number of students to be read is specified as the first command-line argument. Sorting can be done by **name**, **age**, or completed university **credits**. Which one depends on what is specified as the second command-line argument: **name**, **age**, or **credits**. A student is read as a personnummer, a surname, a first name, and a number of university credits.

Here you see the outputs that should be generated when standard input is redirected to the file "**few-students.txt**", which contains:

```
0101019999 Randomsdotter Student 7.5
0202028888 Studentdotter She 15
0302017777 Studentsson He 4.5
0302016666 Randomsson Student 3.0
```

```
java SortStudents 4 name < few-students.txt
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.5/
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
```

```
java SortStudents 4 age < few-students.txt
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.5/
```

```
java SortStudents 4 credits < few-students.txt
[Student]/Randomsson, Student, pn: 0302016666, credits: 3.0/
[Student]/Studentsson, He, pn: 0302017777, credits: 4.5/
[Student]/Randomsdotter, Student, pn: 0101019999, credits: 7.55/
[Student]/Studentdotter, She, pn: 0202028888, credits: 15.0/
```

2. This assignment is about implementing a data type that is quite similar to an array. The big difference is that ***you don't need to determine the size***; instead, the size is determined by the number of elements stored within it. This data type is a so-called data structure: the data type does not correspond to something like a color, an image, complex numbers, students, or other types of objects one might want to use in a program. Instead, the data type is used to store data (***just like arrays do!***).

This data structure is usually called an extendable (or extensible or dynamic) array: one can think of extendable arrays as arrays that can be extended during program execution. Programs that use this data structure are those that need to use arrays, but where it is not known before the program runs how many slots will be needed (typically, many elements are read from a file, or programs where the number of elements needs to change during execution).

In the file ***Xarray.java***, there is the beginning of an implementation of a data type for extendable arrays: arrays that can be extended during program execution.

The data type's API is as follows (more detailed explanations follow in the subtasks):

```
public class Xarray<T>{
-----
// The constructor creates an array of T with 10 slots
// to be able to store the elements.
// The number of elements in the array is 0, but it has space to
// store 10 elements.
public Xarray()

// The add method adds the value x to the first available slot in the array.
// If there isn't enough space
// a larger array is created and existing elements are copied there.
public void add(T x)

// The lookup method returns the value at position i in the array
public T lookup(int i) throws IndexOutOfBoundsException

// The update method changes the value at position i in the array to x
public void update(int i, T x) throws IndexOutOfBoundsException

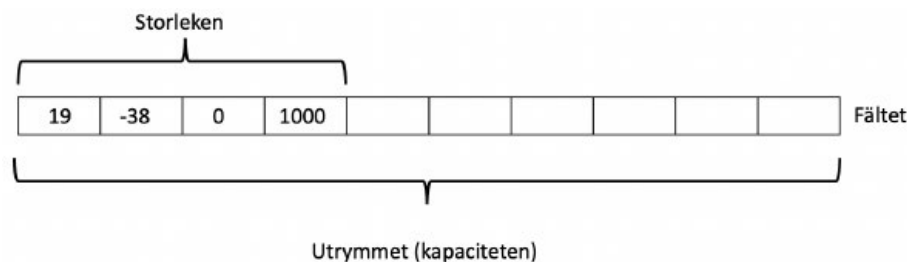
// The size method returns the number of elements in the array (the size)
public int size()
}
```

You should program the Xarray class. To do this, you should:

(a) (1 point) Use three instance variables:

- a variable of type **T[]** (i.e., an ordinary array **data**) to store the elements in **Xarray**,
- a variable of type **int** to keep track of the array's **capacity** (the length of the first variable), and
- a variable of type **int** to keep track of how many elements are currently stored in the array. This is the **size** of the **Xarray**.

The image shows how we interpret these three variables:



(b) (1 point) Program the constructor so that when a client program creates an instance with **`new Xarray()`**, the instance variables are initialized to an array with 10 slots, the size to 0, and the capacity to 10.

(c) (1 point) Program the **`add`** method to add an element. The element should be placed in the first available slot in the array. The first available slot can be calculated using the instance variable that keeps track of the size. When an element is added, the size must be changed (it increases by one). If the method is called when the size is the same as the capacity (there is no available slot for the new element), the implementation should:

- create an array with double the number of slots,
- copy all values in the instance variable (the array) to the new array,
- update the instance variable (the array) to point to the new, longer array,
- update the instance variable for the capacity,
- add the new element to the first available slot, and
- update the instance variable for the size.

(d) (1 point) Program the lookup method so that it returns the value that exists at the position specified in the argument. If the argument is not a position where an element exists (less than 0 or greater than or equal to the size), the method should throw an **`IndexOutOfBoundsException`**.

(e) (1 point) Program the update method so that the value at the specified position receives the value provided. If the position is not a position where an element exists (less than 0 or greater than or equal to the size), the method should throw an **`IndexOutOfBoundsException`**.

(f) (0 points) Program the size method that returns the number of elements in the array.

(g) (1 point) Write a program **`StudentXarray`** that reads students from standard input, creates an **`Xarray`** with these students, and prints the students sorted by completed university credits. The program should be a class named **`StudentXarray`** in a file **`StudentXarray.java`**. The program should not use command-line arguments or other input to determine how many students to read.

You can test the program with the file **`many-students.txt`** where you can redirect the program's standard input.

Tip: Read about `java.util.Scanner` and `System.in`

NOTE! To be able to sort with **`Arrays.sort`** and **`StudentCreditsComparator`**, you will need to create a Java array with all elements from the **`Xarray`** you created with all the input students.

BONUS!!!

Submission Assignment - Part 2

3. (2 points) Complete the main method in the **SandC.java** class, which reads students and anonymization codes from standard input and can produce two outputs:

1. either an output sorted according to the students' surnames
2. or an output sorted according to the anonymization codes

The program **should use** a **command-line argument** to determine how the output should be sorted: either **code or student**.

Students and codes are read as **four Strings**: personnummer, surname, first name, and code.

The Pair class is available in Blackboard, with the material for week 1 of the course.

You should define a **Comparator** for the **Pair** class where the order is primarily determined by the second component of the pair.

Then you can use **GenericSorting.insertionSort** to sort by the first component and **Arrays.sort** with the new Comparator to sort by the second component.

You can test the program by **redirecting standard input** to the **SandC.txt** file, which contains 100 students with anonymization codes.

Verify that the output is sorted as desired according to the command-line argument.